

Documentação de C++: Strings, Structs, Funções e Enums

1. Strings

A classe `std::string` representa sequências de caracteres na Standard Library do C++. Abaixo, alguns métodos básicos:

- **length() / size()**: Retorna o número de caracteres.
- **substr(pos, count)**: Extrai uma substring a partir de `pos` com `count` caracteres.
- **find(str)**: Retorna a posição da primeira ocorrência de `str` ou `string::npos`.
- **rfind(str)**: Posição da última ocorrência.
- **append(str)**: Concatena `str` ao final.
- **c_str()**: Retorna um ponteiro `const char*` para o buffer C-string.
- **empty()**: Retorna `true` se vazia.
- **insert(pos, str)**: Insere `str` em `pos`.
- **erase(pos, count)**: Remove `count` caracteres a partir de `pos`.
- **replace(pos, count, str)**: Substitui `count` caracteres em `pos` por `str`.
- **toupper / tolower**: Conversão de caracteres (requere `e`).

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

/*
 * Exemplo de uso de std::string
 */
int main() {
    string s = "Hello, C++ World!";
    // tamanho da string
    cout << "Length: " << s.length() << endl;

    // encontrar substring
    size_t pos = s.find("C++");
    if (pos != string::npos) {
        cout << "'C++' found at position " << pos << endl;
    }

    // extrair substring
    string sub = s.substr(7, 3); // "C++"
    cout << "Substr: " << sub << endl;

    // converter para maiúsculas
    transform(s.begin(), s.end(), s.begin(), ::toupper);
    cout << "Upper: " << s << endl;

    return 0;
}
```

2. Structs

Em C++, struct define um tipo com múltiplos campos. Pode-se sobrecarregar operadores e diferenciar de std::pair e std::tuple:

```
#include <iostream>
using namespace std;

// Definição de struct Ponto
struct Ponto {
    double x, y;

    // Construtor
    Ponto(double _x = 0, double _y = 0) : x(_x), y(_y) {}

    // Sobrecarga do operador +
    Ponto operator+(const Ponto& other) const {
        return Ponto(x + other.x, y + other.y);
    }

    // Sobrecarga do operador <<
    friend ostream& operator<<(ostream& os, const Ponto& p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
};

int main() {
    Ponto a(1.0, 2.0), b(3.0, 4.0);
    Ponto c = a + b; // usa operator+
    cout << "c = " << c << endl;

    // std::pair e std::tuple
    pair<int, string> par = {1, "um"};
    tuple<int, string, double> tup = make_tuple(2, "dois", 2.0);

    cout << "pair: (" << par.first << ", " << par.second << ")" << endl;
    cout << "tuple: ("
        << get<0>(tup) << ", "
        << get<1>(tup) << ", "
        << get<2>(tup) << ")" << endl;

    return 0;
}
```

3. Funções

Declaração de funções e uso de recursão em C++:

```
// Declaração de função
int soma(int a, int b);

// Definição da função
int soma(int a, int b) {
```

```

        return a + b;
    }

// Função recursiva: fatorial
int fatorial(int n) {
    // Caso base
    if (n <= 1) return 1;
    // Caso recursivo
    return n * fatorial(n - 1);
}

int main() {
    cout << "5! = " << fatorial(5) << endl;
    return 0;
}

```

4. Enumerações (enum)

Define um tipo que pode assumir valores limitados e nomeados:

```

#include <iostream>
using namespace std;

// Definição de enum para dias da semana
enum Dia { Domingo, Segunda, Terca, Quarta, Quinta, Sexta, Sabado };

int main() {
    Dia hoje = Sexta;
    if (hoje == Sexta) {
        cout << "Sexta-feira!" << endl;
    }
    return 0;
}

// Enum class (escopo fechado)
enum class Cor { Vermelho, Verde, Azul };

Cor c = Cor::Verde;

```